

Learning by Discovering Problem Solving Heuristics Through Experience

Wu Wang and Jianhua Chen

Abstract—Learning is the increase of knowledge and improvement of skills. In this paper, we present a system SAFE (Strategy Acquisition From Experience), which incorporates new methods for discovering domain-dependent problem solving heuristics. SAFE is implemented as a sorting system whose sorting strategies are represented as production rules. SAFE initially uses the insertion sort strategy to solve problems. After solving each given problem, SAFE learns symbolic rules from the solution path which is obtained by applying the existing heuristic information. By one or several processes of learning, SAFE is able to obtain the heuristics good enough to sort new problems with minimum exchanges of elements. The optimal heuristics that SAFE learns can direct itself to behave as using the selection sort strategy which has the minimum exchanges of elements during sorting. The notion of “shortcut” is introduced which is an effective inductive learning bias for reducing the hypothesis space to be searched during learning. The learning method in SAFE can be generalized to other areas.

Index Terms—Expert system, inductive learning, learning symbolic rules, machine learning, rule-based system.

I. INTRODUCTION

IT has long been a goal of artificial intelligence (AI) to develop computer systems that could be taught rather than programmed and intelligent programs that could acquire new knowledge [1], [2], [4], [5]. In this paper, we present a learning system SAFE which learns problem solving heuristics through experience. Currently, most of the expert systems do not have the built-in ability to learn new heuristics so as to improve their performance. The thrust of our research is to integrate learning ability into expert systems to improve their problem solving ability.

Learning can be defined as the process by which a system improves its performance with practice and acquires knowledge on its own. The ability to search is believed to be central to intelligence [6]. The ability to direct search down profitable paths is what distinguishes the expert from the novice. Thus, learning is the process of acquiring profitable searching strategies. Fig. 1 gives a simple model of learning systems. Our system, SAFE (Strategy Acquisition from Experience), follows this model.

A built-in learning element in an expert system is able to examine the expert system's past problem solving experience and learn new knowledge to enhance the expert system's

performance. The major learning technique employed by the learning element is inductive learning, i.e., learning from examples. The quality of information given to the learning element can have a significant effect on the difficulty of the learning task. The representation and content of the knowledge base is another important factor in the design of learning systems. The representational forms vary along four important dimensions: expressiveness, ease of inference, modifiability, and extendibility. Only production rule systems are considered in this paper since production rules have good properties in these four dimensions. It is necessary for inductive learning system to coordinate some domain-dependent knowledge and general learning strategies to gain good performance. SAFE incorporates three domain-dependent primitives “>”, “<”, and “exchange” with its general learning mechanism.

The learning element works corporatively with the performance element. The new rules learned must be applied to the problem solving task by the performance element and the feedback from the performance element may suggest further modification of the rules. The problem of consistency check and knowledge base maintenance must also be considered. The third issue is the transparency of the performance element. It is useful for the learning element to have access to the internal actions of the performance element. To make the performance element transparent to the learning element, the performance element of SAFE provides the solution path explicitly to the learning element.

The notion of “shortcut” is introduced which means a less-cost subchain in a solution path obtained by an expert system's problem solving process. The shortcut is a very powerful inductive learning bias in reducing the size of hypothesis space searched by the learning element.

There are several related works in the area of learning to acquire problem solving skills within expert problem solvers. Korf [7] presented a method to learn macro-operators in a decomposable problem domain. For problems like Rubik's cube, eight puzzles, a macro table can be created by Korf's method using a search-based preprocessing phase and thus the solutions can be efficiently found from any initial state to a particular goal state. Our learning method of generating new rules by following shortcuts can be regarded as generating a new macro operator in Korf's terminology. However, our method is more general, in that for problems with large number of initial states and goal states, the use of macro table is impractical, while one can still use our method to create new rules which are an abstract form of “initial state–goal state” description. Our learning method also bears similarity with the

Manuscript received April 17, 1989; revised December 22, 1989.

W. Wu is with Metaphor Computer Systems, Mountain View, CA 94043.

J. Chen is with the Department of Computer Science, Louisiana State University, Baton Rouge, LA 70803

IEEE Log Number 9101538.

1041-4347/91\$01.00 © 1991 IEEE

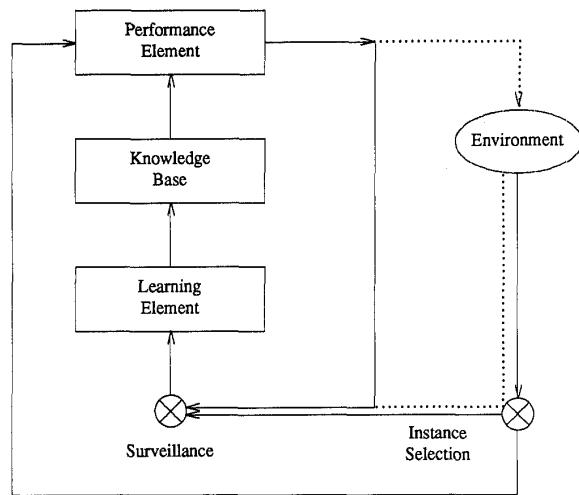


Fig. 1. A simple model of learning systems.

“chunking” technique in SOAR [8]. The difference between our work and that in SOAR is that we do not form a chunk each time a goal is satisfied as is the case in SOAR. The learning element in our system learns new rules only when it finds a “shortcut” in a solution path which suggests an improvement of the expert knowledge can be obtained by following the shortcut.

The paper is organized as follows. Section II gives the learning task and the application field of SAFE. The learning process of SAFE is described in detail in Section III and Section IV. We discuss the information collection in Section III. In Section IV, the analysis of information and the construction of new production rules are illustrated.

II. LEARNING TASK

The thrust of our research is to develop techniques that improve expert system performance during problem solving. The paradigm that human experts restudy the solutions after solving problems and learn from it is incorporated.

The problem domain is selected as the sorting problem. A sorting program takes as input of n arbitrary distinct integers $A(1), A(2), \dots, A(n)$, and produces an ordered sequence $(A'(1), A'(2), \dots, A'(n))$ which is a permutation of $(A(1), A(2), \dots, A(n))$ such that $A'(1) < A'(2) < \dots < A'(n)$. Fig. 2 is a model of sorting systems.

There are many sorting algorithms. The insertion strategy is the simplest and the selection sorting algorithm is one of the algorithms with minimum number of element exchanges.

SAFE initially has the insertion sort ability. It is designed to have the ability to learn from experience after it solves problems. The goal of learning is to obtain good strategies for sorting to minimize the exchanges of elements, not necessarily the same as the selection algorithm. All heuristic information is in the production rule form as the following

((conditions) (action))

where (conditions) has only two binary primitives, “>” and

“<,” and (action) has only one binary primitive, “exchange”.

Initially, SAFE has only one rule representing the insertion sort method. The rule is

$((A(i) > A(i+1)) (\text{exchange } A(i)A(i+1)))$.

III. NOTION OF SHORTCUT AND INFORMATION COLLECTION

The learning element in our focus learns from the problem solving examples which are solution paths produced by the expert system. Typically, the solution of a problem is represented as a path from initial state to the goal state. Therefore, each node in a solution path produced by the expert system represents a state and the edge from one node to the next one in the chain represents the application of an operator. The learning element is to examine these solution paths to extract new rules which will reduce the amount of search in the expert system’s problem solving. The instance space for the learning element consists of all the possible solution paths and the hypothesis space consists of all the possible rules which can be generated from these solution paths.

In practical learning, the instance space and hypothesis space will be very large. If no inductive bias is employed to restrict the focus of learning, any inductive method can do no better than random guessing. So we introduce the notion of “shortcut” as our inductive bias and we require that only the hypotheses derived from examples (solution paths) containing “shortcut” are to be examined.

The notion shortcut is related to the concept of optimal solution path. Assume that for a solution path P , each edge in P is associated with a cost value, and the cost of P is the sum over all edges in P . An optimal solution path for a problem is a path with lowest cost value between the given initial state and goal state. A shortcut in a solution path P is a link between two nodes n_i, n_j , where $i < j$, such that the cost of this link from n_i to n_j is less than the cost of the subpath $(n_i, n_i+1, \dots, n_j)$ in P . In SAFE, we use an operator “shortcut” to detect the shortcuts from a solution path in the information collection phase of learning.

Information collection is the first step of learning. The collected information quality and level have a significant effect on the learning task. Only low-level information is collected for SAFE. SAFE only collects “noise-free” information. Thus, SAFE cannot collect any information from the solutions of some problems. SAFE would rather not learn than learn something wrong from low-quality information.

For an input problem, SAFE solves the problem using the existing heuristics which are the initial insertion sort strategy or some other rules learned from previous examples. A solution path is obtained after solving the problem. Since SAFE is implemented in Franz Lisp on the VAX 11/780, the input to SAFE is a list of integers, not necessarily being positive, in the form such as (3 2 5 8). Thus, the solution path is a sequence of lists, where the first list is the input and the last list is in ascending order. Except for the first list, each list in the solution path is obtained by exchanging two elements of the immediately preceding list. For a problem of (4 2 3 1 5 6), one solution can be seen in Fig. 3.

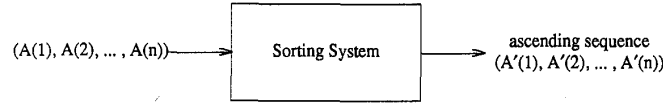


Fig. 2. A model of sorting systems.

((4 2 3 1 5 6) (2 4 3 1 5 6) (2 3 4 1 5 6) (2 3 1 4 5 6) (2 1 3 4 5 6) (1 2 3 4 5 6))

Fig. 3. One solution for input (4 2 3 1 5 6).

The solution in Fig. 3 needs 5 exchanges of elements. For one problem, there are many solution paths leading to the goal. Some solution paths are lower cost and others are higher cost. A problem solving system is to search for a low cost, usually not the lowest cost, solution path in short time. The solution finding time is expected to be short since it is also a cost factor. In SAFE, the exchanges of elements are considered the cost of the solution. SAFE tries to reduce the solution cost by finding better heuristics to direct the search. Many previous learning systems, like Langley's [3], first find many solution paths for input problems and then learn from it. Those methods are considered not practical since it is time-consuming and storage-consuming, particularly for application systems with some heuristics already to learn during problem solving to refine their existing heuristics. We believe that learning from solution paths is more practical in applications since the solution paths are automatically obtained after solving the problems using the existing heuristics.

SAFE is to collect information from the solution paths. If a solution path is bad, there are often some shortcuts on the solution path. Notice that this is not always true. If there are no shortcuts on the solution path, which may be or may not be optimal, SAFE is not able to collect any information for learning. Fig. 4 gives an example. These cases usually have "no too bad" solutions relative to optimal ones.

That SAFE is designed in this way is very similar to that human beings learn from experience. Human experts often restudy the solutions after solving the problems to find some dumbness, to learn from it, and to direct future problem solving.

The shortcut finding algorithm in SAFE works as follows. For each pair of nonadjacent nodes n_i, n_j in the solution path, where $i < j$, check whether n_i and n_j differ exactly at two positions, say $A(l)$ and $A(k)$ in node n_i are different from $A'(l)$ and $A'(k)$ in node n_j . If so, node n_j can be obtained by exchanging elements $A(l)$ and $A(k)$ in node n_i . Note that if n_i and n_j differ at more than two positions, the algorithm knows there is no shortcut from n_i to n_j .

For the example in Fig. 3, which is also obtained by using the insertion sort heuristics, there are two shortcuts on the solution path. Fig. 5 illustrates the shortcuts on the solution path.

From *shortcut 1* SAFE collects a set of data shown in Table I using the three known primitives, "<", ">" and "exchange". From *shortcut 2* SAFE collects another set of data shown in Table II. Each datum is "weighted" according to how many steps are skipped. The weights of the data are part of the information for credit assignment of new rules.

((3 5 2 1) (3 2 5 1) (2 3 5 1) (2 3 1 5) (2 1 3 5) (1 2 3 5))

Fig. 4. An example of solution paths without shortcuts.

IV. INDUCTION AND RULE CONSTRUCTION

After the data are collected, SAFE is to extract useful heuristics from the data. Since the data collected are too specific and detailed, SAFE must hypothesize more general rules. This process is called inductive learning, which can be viewed as search for plausible general descriptions (inductive assertions) that explain the collected data from the given input examples and are useful for predicting new data for the new problems.

It is a common view that general methods of formal induction, though mathematically elegant and theoretically applicable to many problems, are in practice very inefficient and rarely lead to interesting solutions. Some researchers have concentrated on learning problems in some specific domains. Such an approach can produce novel and practical solutions. On the other hand, it is difficult to extract general principles of induction from such problem-specific work. It is thus difficult to apply such special-purpose programs to new areas.

SAFE uses only three domain-dependent primitives, ">", "<" and "exchange". The methodology used in SAFE can be applied to new areas. SAFE first hypothesizes a general rule for every datum in the data set collected by the information collection element. An example is shown in Fig. 6. Condition $(A(i \text{ to } (j-1)) > A(j))$ stands for $A(i) > A(j), A(i+1) > A(j), A(i+2) > A(j), \dots, A(j-1) > A(j)$ and condition $(A(i-) > A(j))$ stands for $A(1) > A(j), A(2) > A(j), \dots, A(i-1) > A(j)$, and the condition $(A(j+) > A(j))$ stands for $A(j+1) > A(j), A(j+2) > A(j), \dots, A(n) > A(j)$, where i is less than j .

Since for a permutation of $\{1, 2, \dots, n\}$ a hypothesis rule shown in Fig. 6 have multiple choices of i and j which can satisfy the condition of the hypothesis rule, SAFE also needs to hypothesize the policies of selecting i and j if a hypothesis rule is applicable in some situations. From the example in Fig. 5, SAFE obtains a policy of selecting i and j as $(\text{maximize}(j-i))$ which is obtained by analyzing the distances of the exchanged elements and the shortcuts. In fact, SAFE is always to find the policy of $(\text{maximize}(j-i))$ for the sorting problem. Thus, we omit the policies of selecting i and j hereafter although every hypothesis rule is factually associated with one policy in SAFE. From the data sets in Table I and Table II, SAFE hypothesizes a set of general rules shown in Table III.

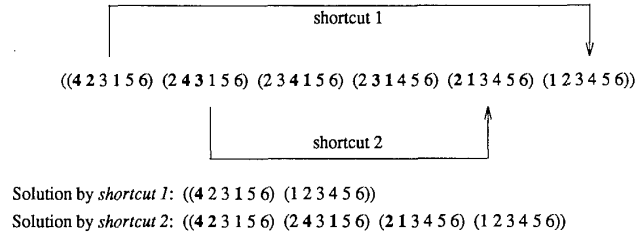


Fig. 5. Shortcuts on the solution path.

TABLE I
DATA FROM *shortcut 1*

Weight	Rule
6	$((A(1) > A(4))(A(2) > A(4))(A(3) > A(4)))$ (exchange $A(1)A(4)$)
6	$((A(4) < A(1))(A(3) < A(1))(A(2) < A(1)))$ (exchange $A(1)A(4)$)
6	$((A(6) > A(1))(A(5) > A(1))(A(4) < A(1))(A(3) < A(1))(A(2) < A(1)))$ (exchange $A(1)A(4)$)

TABLE II
DATA FROM *shortcut 2*

Weight	Rule
4	$((A(2) > A(4))(A(3) > A(4)))$ (exchange $A(2)A(4)$)
4	$((A(4) < A(2))(A(3) < A(2)))$ (exchange $A(2)A(4)$)
4	$((A(6) > A(2))(A(5) > A(2))(A(4) < A(2))(A(3) < A(2)))$ (exchange $A(2)A(4)$)
4	$((A(1) > A(4))(A(2) > A(4))(A(3) > A(4)))$ (exchange $A(2)A(4)$)
4	$((A(6) > A(2))(A(5) > A(2))(A(4) < A(2))(A(3) < A(2))(A(1) < A(2)))$ (exchange $A(2)A(4)$)
4	$((A(1) > A(4))(A(2) > A(4))(A(3) > A(4))(A(5) > A(4))(A(6) > A(4)))$ (exchange $A(2)A(4)$)

Datum: $((A(1) > A(4))(A(2) > A(4))(A(3) > A(4)))$ (exchange $A(2)A(4)$)

Hypothesis 1: $((A(i \text{ to } j-1) > A(j))(A(i-1) > A(j)))$ (exchange $A(i)A(j)$)

Hypothesis 2: $((A(i \text{ to } j-1) > A(j)))$ (exchange $A(i)A(j)$)

Fig. 6. An example of hypotheses from data.

SAFE is then to analyze the hypothesis rules to determine which of them are more heuristic and which of them are not useful or wrong. The knowledge base must be consistent after adding new rules. That is, new rules must be assigned credits or weights. There are many ways to assign credits. New rules might be applied back to the same example to test the performance; new rules might be checked against the existing rules and so on. SAFE analyzes the new rules according to the policies associated with the rules. Some useless rules shown in Table IV are deleted from the hypothesis rules in Table III. The useless rules are detected by examining whether they follow the above policy of maximizing the distance of the elements to be exchanged. No conflicting hypothesis rules are obtained for the sorting problem.

SAFE assigned weights to rules according to how many heuristic conditions in the rules and the weights of data from which rules are hypothesized, since all conditions in rules are useful after useless rules are deleted. If a rule is relearned, its new weight is added to its existing weight. Table V shows the new rules learned from the example in Fig. 3.

The rule

$$(((A(i-1) < A(i))(A((i+1) \text{ to } j)$$

$$< A(i))(A(j+1) > A(i)))$$
 (exchange $A(i)A(j)$)

is awarded the highest weight and is to direct the performance element to work with minimum exchanges of elements. The performance element picks the highest weighted rule to use first. If the rule is not applicable, then the performance element selects the next highest weighted rule and so forth. Since the first rule in Table V is always applicable before the goal is reached, it directs the performance element to work as using the selection sort strategy. In fact, if the first input problem to SAFE is (1 4 3 2 5), SAFE learns a set of new rules shown in Table VI, where the first rule is exactly the same as the aforementioned selection sort algorithm.

After learning the best rule to direct the performance element, SAFE will not be able to collect information for learning any more since the solution is optimal. In this situation, the learning element can be turned off by the user or by SAFE itself.

V. CONCLUSION

Machine learning has long been a goal of AI. Machine learning research strives to open the possibility of instructing

TABLE III
HYPOTHESIS RULES FROM TABLE I AND TABLE II

Weight	Rule
4	$((A(i-1) < A(i))(A((i+1) \text{ to } j) < A(i))(A(j+1) > A(i))) \text{ exchange } A(i)A(j))$
10	$((A((i+1) \text{ to } j) < A(i))(A(j+1) > A(i))) \text{ (exchange } A(i)A(j))$
4	$((A(j+1) > A(j))(A(i \text{ to } (j-1)) > A(j))(A(i-1) > A(j))) \text{ (exchange } A(i)A(j))$
4	$((A(i \text{ to } (j-1)) > A(j))(A(i-1) > A(j))) \text{ (exchange } A(i)A(j))$
10	$((A(i \text{ to } (j-1)) > A(j)) \text{ (exchange } A(i)A(j))$
10	$((A((i+1) \text{ to } j) < A(i)) \text{ (exchange } A(i)A(j))$

TABLE IV
RULES ARE CONSIDERED USELESS AND ARE DELETED FROM TABLE III BY SAFE

$((A(j+1) > A(j))(A(i \text{ to } (j-1)) > A(j))(A(i-1) > A(j))) \text{ (exchange } A(i)A(j))$
$((A(i \text{ to } (j-1)) > A(j))(A(i-1) > A(j))) \text{ (exchange } A(i)A(j))$

TABLE V
NEW RULES LEARNED FROM TABLE I AND TABLE II

Weight	Rule
88	$((A(i-1) < A(i))(A((i+1) \text{ to } j) < A(i))(A(j+1) > A(i))) \text{ (exchange } A(i)A(j))$
34	$((A((i+1) \text{ to } j) < A(i))(A(j+1) > A(i))) \text{ (exchange } A(i)A(j))$
10	$((A(i \text{ to } (j-1)) > A(j)) \text{ (exchange } A(i)A(j))$
10	$((A((i+1) \text{ to } j) < A(i)) \text{ (exchange } A(i)A(j))$

TABLE VI
RULES LEARNED FROM THE SOLUTION PATH FOR PROBLEM (1 4 3 2 5)

Weight	Rule
104	$((A(j+1) > A(j))(A(i \text{ to } (j-1)) > A(j))(A(i-1) < A(j))) \text{ (exchange } A(i)A(j))$
104	$((A(i-1) < A(i))(A((i+1) \text{ to } j) < A(i))(A(j+1) > A(i))) \text{ (exchange } A(i)A(j))$
24	$((A((i+1) \text{ to } j) < A(i))(A(j+1) > A(i))) \text{ (exchange } A(i)A(j))$
24	$((A(i \text{ to } (j-1)) > A(j))(A(i-1) < A(j))) \text{ (exchange } A(i)A(j))$
4	$((A(i \text{ to } (j-1)) > A(j)) \text{ (exchange } A(i)A(j))$
4	$((A((i+1) \text{ to } j) < A(i)) \text{ (exchange } A(i)A(j))$

computers in new ways, and thereby promise to ease the burden of hand-programming growing volumes of increasingly complex information into the computers of tomorrow.

In this paper, we present a system SAFE which learns problem solving heuristics from experience. SAFE initially has a weak heuristic and learns from solution paths after solving problems. The learning element is efficient and effective. It is able to learn very powerful heuristics from simple examples. The learned rules direct the sorting process to achieve optimal solution path (with minimal number of element exchanges).

This work shows the feasibility to integrate a learning element in an expert system. The experimentation confirms the validity of our learning method for improving expert systems' performance.

The important notion of "shortcut" is introduced which is in general a suitable inductive learning bias for a learning element in an expert system. The rules learned from a shortcut are guaranteed to be noise-free. The learning methodology in SAFE is applicable to new areas. In general, it is desirable to integrate the concept of learning in expert systems to improve the system's performance.

ACKNOWLEDGMENT

SAFE is based on W. Wang's independent study work

under the supervision of Dr. S. Kundu. W. Wang would like to thank Dr. S. Kundu for his suggesting the sorting problem to be tackled and Dr. S. S. Iyengar for his comments on early versions of this paper. The authors are grateful to the reviewers for their invaluable comments.

REFERENCES

- [1] P. R. Cohen and E. A. Feigenbaum, *The Handbook of Artificial Intelligence*, Vol. 3. Los Altos, CA: Kaufmann, 1982.
- [2] J. H. Holland, K. F. Holyoak, R. E. Nisbett, and P. R. Thagard, *Induction: Processes of Inference, Learning, and Discovery*. Cambridge, MA: M.I.T. Press, 1986.
- [3] P. Langley, "Learning to search: From weak methods to domain-specific heuristics," *Cognitive Sci.*, no. 9, pp. 217-260, 1985.
- [4] R. S. Michalski, J. G. Carbonel, and T. M. Mitchell, *Machine Learning: An Artificial Intelligence Approach*. Palo Alto, CA: Tioga, 1983.
- [5] T. M. Mitchell, J. G. Carbonel, and R. S. Michalski, Eds., *Machine Learning: A Guide to Current Research*. Norwell, MA: Kluwer Academic, 1986.
- [6] A. Newell and H. A. Simon, "Computer science as empirical inquiring: Symbols and search," in *ACM Turing Award Lectures: The First Twenty Years*, ACM Press, 1987.
- [7] R. E. Korf, "A program that learns to solve Rubik's cube," *Proc. AAAI*, 1982.
- [8] P. S. Rosenbloom *et al.*, "Current research on learning in SOAR," in *Machine Learning: A Guide to Current Research*, T. Mitchell, J. Carbonel, and R. Michalski, Eds. Norwell, MA: Kluwer Academic, 1986.



Wu Wang received the B.S. degree in computer science from Zhongshan University, Canton, China, in 1984. He joined the Department of Computer Science, Louisiana State University, Baton Rouge, in 1987, for his graduate study and received the Ph.D. in 1989.

He is currently with Metaphor Computer Systems, Mountain View, CA. He has performed research on robot motion planning, speech recognition, natural language processing, robot vision, and machine learning. His technical papers have

appeared in computer science journals and conferences.

Jianhua Chen received the B.S., M.S., and Ph.D. degrees in computer science from Jilin University, Changchun, the People's Republic of China, in 1982, 1985, and 1988, respectively.

She was with China Software Technique Corporation, Beijing, China, in the spring of 1988 before she joined the Department of Computer Science, Louisiana State University in August 1988. She is currently as Assistant Professor in computer science at LSU. She has worked in the areas of symbolic computation algorithms, nonmonotonic logic in artificial intelligence, machine learning, database and knowledge-based systems, and distributed computing algorithms. Her research papers have appeared in technical journals and conference proceedings.